

Designing and Developing a Modern Developer Portfolio with Next.js

Architecture, Animations, Performance, and Full-Stack Integration
of a Production-Grade Personal Portfolio Website

Gaurav Ganesh Teegulla

B.Tech Computer Science (Data Science)
Sreenidhi Institute of Science and Technology, Hyderabad, India
gauravganeshteegulla@gmail.com

June 2026

Abstract

In the current competitive technology landscape, a developer's personal portfolio website serves as the primary medium for showcasing technical competence, aesthetic sensibility, and professional identity. This paper presents the end-to-end design and development of a modern, production-grade personal portfolio website built using Next.js 14 with the App Router, TypeScript, Tailwind CSS, and Framer Motion. The system integrates a full-stack email contact pipeline via the Resend API, implements dark/light theming through CSS custom properties, and delivers smooth, hardware-accelerated animations through scroll-triggered Framer Motion variants. The architecture prioritises performance through static generation, lazy loading, and responsive image optimisation. A systematic component hierarchy — spanning Hero, About, Skills, Experience, Projects, Education, Certifications, and Contact sections — is documented alongside implementation decisions, trade-offs encountered, and lessons learned. The final artefact achieves a Lighthouse performance score exceeding 90, zero TypeScript compilation errors, and a fully functional server-side contact delivery system — demonstrating that a polished, maintainable portfolio can be engineered using entirely open-source tooling and deployed globally for zero cost.

Keywords: Next.js, TypeScript, Tailwind CSS, Framer Motion, Portfolio, Web Performance, Dark Mode, Resend API, Vercel Deployment, Responsive Design

Table of Contents

1. Introduction

- 1.1 Motivation and Background
- 1.2 Research Objectives
- 1.3 Scope and Limitations

2. Literature Review

- 2.1 Evolution of Developer Portfolios
- 2.2 Modern Frontend Frameworks
- 2.3 Animation Libraries in Web Development
- 2.4 Theming Approaches

3. System Architecture and Design

- 3.1 Technology Stack Selection
- 3.2 Component Hierarchy
- 3.3 Routing and Page Structure
- 3.4 Design Token System

4. Implementation

- 4.1 Project Initialisation
- 4.2 Navigation and Loading
- 4.3 Hero and About Sections
- 4.4 Skills Section
- 4.5 Projects Section
- 4.6 Experience and Education
- 4.7 Certifications and Achievements
- 4.8 Contact Form and Email Pipeline

5. Animation and Motion Design

- 5.1 Framer Motion Architecture
- 5.2 Scroll-Triggered Reveals
- 5.3 Loader Animation
- 5.4 Micro-interactions

6. Theming: Dark and Light Modes

- 6.1 CSS Custom Properties Strategy
- 6.2 next-themes Integration
- 6.3 Component-Level Token Usage

7. Performance Optimisation

- 7.1 Static Generation
- 7.2 Image Optimisation

7.3 Font Loading

7.4 Bundle Analysis

8. Deployment and DevOps

8.1 Vercel Deployment Pipeline

8.2 Environment Variables

8.3 Continuous Deployment via GitHub

9. Testing and Quality Assurance

10. Results and Discussion

11. Conclusion and Future Work

12. References

1. Introduction

1.1 Motivation and Background

The digital presence of a software engineer has become as consequential as their resume. Recruiters, collaborators, and potential clients increasingly evaluate developers not only on the basis of their academic credentials or years of experience, but on the quality, accessibility, and aesthetic coherence of the work they choose to present publicly. A well-crafted portfolio communicates technical depth, product thinking, and attention to detail simultaneously — qualities that a PDF resume cannot convey.

This project originated from a desire to build a portfolio that transcends the typical template-driven approach. Rather than deploying a pre-built theme or a drag-and-drop website builder, the goal was to engineer a bespoke, performant, and visually compelling single-page application from first principles using modern open-source tooling. Every design decision, animation curve, and colour token was deliberately chosen.

The reference site for structural inspiration was Brittany Chiang's acclaimed portfolio (v4.brittanychiang.com), widely cited in the developer community as a gold standard for clean, minimal, and functional portfolio design. The final implementation, however, diverges significantly in its technology choices, visual identity, and feature set — adapting the structural philosophy to reflect the author's own project portfolio and aesthetic preferences.

1.2 Research Objectives

This paper documents the following objectives:

- To analyse the structural and experiential requirements of a modern developer portfolio.
- To design a component architecture that is maintainable, extensible, and semantically correct.
- To implement a full-stack Next.js 14 application with server-side API routes.
- To integrate smooth, accessible animations using Framer Motion without sacrificing performance.
- To build a robust dark/light theming system using CSS custom properties and next-themes.
- To wire a production-ready contact form that delivers emails via the Resend API.
- To deploy the site globally using Vercel's edge network with zero marginal cost.

1.3 Scope and Limitations

The scope of this paper is limited to the architecture and implementation of the frontend portfolio application and its contact API route. It does not cover the individual projects featured within the portfolio (VyroCoding, VyroMusic, VyroPortify) in technical depth — each of those is a separate system deserving its own documentation. The paper also does not perform a formal

user study or collect empirical usability metrics; performance observations are drawn from Lighthouse tooling rather than controlled experimentation.

2. Literature Review

2.1 Evolution of Developer Portfolios

The developer portfolio has evolved through three distinct generations. The first generation (circa 2000–2010) consisted of static HTML pages, often hosted on personal domains or university web servers, displaying a name, a list of technologies, and links to downloadable ZIP files. The second generation (2010–2018) saw the widespread adoption of WordPress, Squarespace, and template-based builders, democratising portfolio creation but at the cost of differentiation. The third and current generation (2018–present) is characterised by framework-driven, component-based single-page applications deployed on global CDNs, often featuring WebGL backgrounds, physics-based animations, and integrated CMS pipelines.

Research by Dillahunt et al. (2018) on digital self-presentation in technical hiring found that recruiters spend an average of 6–8 seconds on initial evaluation of a portfolio landing page, prioritising visual clarity, load time, and mobile responsiveness above content depth in the first pass. This finding directly influenced the architectural decision to invest heavily in the above-the-fold Hero section, the animated loading sequence, and the mobile hamburger navigation.

2.2 Modern Frontend Frameworks

The JavaScript ecosystem has converged on React as the dominant component model for interactive user interfaces [Octoverse, 2024]. Within the React ecosystem, Next.js has emerged as the de facto production framework, providing file-system routing, server components, static generation, and a unified deployment target in Vercel. The Next.js 14 App Router — released in stable form in 2023 — represents a significant architectural shift toward React Server Components (RSC) and nested layouts, enabling fine-grained control over server/client boundaries.

TypeScript has become the industry standard for large-scale JavaScript development, with the 2024 Stack Overflow Developer Survey reporting 78% of professional developers using it. Its static type system catches entire categories of runtime errors at compile time, and its IDE integration — particularly with VS Code — provides intelligent autocompletion that accelerates development velocity.

Tailwind CSS introduced a utility-first paradigm that has proven particularly effective for component-based UI development. Rather than writing custom CSS class names and managing specificity conflicts, Tailwind's atomic utility classes compose directly in JSX, keeping styles co-located with structure. The JIT (Just-In-Time) compiler introduced in Tailwind v3 ensures only used utilities are included in the final bundle.

2.3 Animation Libraries in Web Development

Web animation has traditionally been achieved through CSS transitions, CSS animations, and the Web Animations API. However, for complex orchestrated sequences — such as staggered letter reveals, scroll-triggered section entries, and physics-based spring animations — JavaScript animation libraries provide significantly more expressive power.

Framer Motion, developed by Framer and released as an open-source library in 2019, provides a declarative React API for animations built on top of the Web Animations API and CSS transforms. Its key abstractions — motion components, variants, `AnimatePresence`, and `useInView` — map cleanly onto React's component model and support server-side rendering without hydration mismatches. Performance profiling by the Framer team demonstrates that transform-only animations (translate, scale, rotate, opacity) execute entirely on the GPU compositor thread, achieving consistent 60fps even on mid-range mobile devices.

2.4 Theming Approaches

Dark mode support has become a baseline expectation for developer-facing tools and portfolios. The three primary implementation strategies are: (1) CSS custom properties (variables) toggled via a class on the root element, (2) CSS media query `prefers-color-scheme` for system-level detection, and (3) JavaScript-driven style injection. The `next-themes` library by Pacocoursey provides a production-hardened solution that combines all three approaches — detecting system preference, persisting user choice in `localStorage`, and applying a class attribute to the document root — while avoiding flash-of-incorrect-theme (FOIT) on page load through an injected script tag.

3. System Architecture and Design

3.1 Technology Stack Selection

The technology stack was selected based on four criteria: ecosystem maturity, deployment simplicity, developer experience, and performance characteristics. Table 1 summarises the final selections and the rationale for each choice.

Layer	Technology	Version	Rationale
Framework	Next.js	14.x	App Router, RSC, static generation, Vercel-native
Language	TypeScript	5.x	Type safety, IDE support, zero runtime cost
Styling	Tailwind CSS	3.x	Utility-first, JIT, co-located styles
Animation	Framer Motion	11.x	Declarative, React-native, GPU-accelerated
Theming	next-themes	0.3.x	FOIT-free, localStorage persistence
Email	Resend SDK	3.x	Server-side, branded HTML, free tier
Deployment	Vercel	—	Edge CDN, GitHub CI/CD, zero-config
Icons	Lucide React	—	Tree-shakeable SVG icon library

Table 1: Technology stack with selection rationale.

3.2 Component Hierarchy

The application follows a strict single-page architecture (SPA) with all navigation implemented as smooth-scroll anchor links. The root layout wraps the entire application in a ThemeProvider, ensuring theme state persists across the component tree without prop-drilling. The page component composes all sections in a single render tree, enabling Intersection Observer-based animations to work across a continuous scroll surface rather than across navigation events.

The component hierarchy follows a three-tier structure: (1) Layout components (Nav, Footer, SocialSidebar, EmailSidebar, Loader, Providers) manage application shell concerns; (2) Section components (Hero, About, Skills, Experience, Projects, Education, Certifications, Contact) own their data and animation logic; (3) UI primitives (SectionHeading, AnimatedSection, SocialIcons) provide reusable, composable building blocks.

```
src/
├── app/
│   ├── api/contact/route.ts # Resend email endpoint
│   ├── layout.tsx # Root layout + metadata
│   └── page.tsx # Section composition
├── components/
│   ├── layout/ # Shell components
│   ├── sections/ # Page sections
│   ├── ui/ # Reusable primitives
│   └── data/
```

```
■■■ portfolio.ts # Centralised content store
```

3.3 Routing and Page Structure

Next.js 14's App Router enables co-location of server and client components within the same route segment. For this portfolio, the root `page.tsx` is a Server Component that statically renders the full page markup on build, while individual section components are Client Components ('use client') where interactivity or browser APIs are required. This hybrid approach maximises both initial load performance (SSG) and interactive richness (client animations).

The contact API route (`/api/contact`) is a server-side Route Handler that runs in a Node.js runtime, enabling the use of the Resend SDK with a secret API key that never reaches the client bundle. This is architecturally preferable to the EmailJS approach (client-side email dispatch) because it keeps credentials server-side, enables server-side input validation, and allows for HTML email templating without CORS concerns.

3.4 Design Token System

A centralised design token system implemented via CSS custom properties governs all colour, spacing, and typography decisions. Tokens are declared in the `:root` selector for the default dark theme and overridden in the `.light` class for the light theme. This approach provides a single source of truth for design decisions and enables instantaneous theme switching via a class toggle on the document element.

```
/* Dark theme (default) */
:root {
  --bg-primary: #0d0d1a;
  --bg-card: #14142a;
  --border: #1e1e3f;
  --text-primary: #e8e8f0;
  --text-secondary: #a0a0c0;
  --accent: #7c3aed;
}

/* Light theme override */
.light {
  --bg-primary: #f8f8ff;
  --bg-card: #ffffff;
  --text-primary: #1a1a2e;
  --accent: #7c3aed;
}
```

4. Implementation

4.1 Project Initialisation

The project was initialised using create-next-app with TypeScript, Tailwind CSS, ESLint, and the App Router enabled. The default Next.js configuration was extended to support custom image domains (for external project screenshots) and to configure the Geist font family from Vercel's font distribution.

```
npx create-next-app@latest gaurav-portfolio \
  --typescript --tailwind --eslint --app --src-dir
```

4.2 Navigation and Loading

The navigation component (Nav.tsx) implements several production-quality behaviours: scroll-aware shrinking (the nav background adds blur and reduces opacity at the top of the page, then solidifies on scroll), active section tracking via Intersection Observer through a custom useScrollSpy hook, a mobile hamburger menu with AnimatePresence-driven slide-down animation, and a theme toggle button wired to next-themes.

The loading animation (Loader.tsx) renders the full name 'Gaurav Ganesh Teegulla' as individual characters, each animated with a staggered entry (opacity 0→1, y-translate 40→0, blur-filter 8px→0) with a 50ms inter-character delay. A progress bar and welcome message complete the sequence. The Loader is mounted conditionally in the root layout and removed from the DOM after the animation completes via a setTimeout-driven state change.

4.3 Hero and About Sections

The Hero section renders above the fold and must make an immediate impression. It features a staggered text reveal sequence: the overline ('Hi, my name is'), the primary name heading, the title/role subtitle, the bio paragraph, and the CTA buttons each animate in sequentially with 0.1s delays, creating a cascading entry that draws the eye through the content hierarchy.

The About section presents a prose biography alongside a profile card featuring the author's initials rendered in a violet-to-indigo gradient circle, a statistics row (15+ Projects, 27 Repos), and a decorative offset border element. The prose itself is broken into three paragraphs covering background, technical range, and project philosophy — designed to be scannable in under 20 seconds.

4.4 Skills Section

The Skills section presents six technology categories — Languages, Frontend, Backend, AI/ML, Databases, and Tools & DevOps — each as a distinct card with a unique colour identity. A summary pill bar at the top provides an at-a-glance overview of all categories and their item counts. Each card features a gradient background, colour-coded tags matching the card's accent colour, and a corner glow effect that activates on hover.

4.5 Projects Section

The Projects section divides into two sub-sections: three Featured Projects displayed in an alternating side-by-side layout (screenshot + description card), and a grid of Other Noteworthy Projects displayed as minimal folder-style cards.

Featured project screenshots were captured programmatically using Puppeteer, a headless Chromium automation library. Each project was started on a dedicated localhost port, and a screenshot was taken at 1440x900 viewport resolution after a 3-second stabilisation delay to allow JavaScript-driven content to render. The resulting PNG files were placed in the `public/projects/` directory and served via Next.js's `next/image` component with automatic WebP conversion and lazy loading.

The screenshot overlay effect — a violet tint (`bg-violet-900/40` with `mix-blend-multiply`) that fades out on hover — directly references the visual treatment used on Brittany Chiang's portfolio, creating a sense of depth and directing attention toward the description card when idle and toward the screenshot on interaction.

4.6 Experience and Education

The Experience section presents internship and work history in a timeline-style layout. Each entry features a company name (linked), role title, duration, and bulleted achievements written in the XYZ formula (Accomplished X as measured by Y by doing Z). The Education section uses a similar card structure with institution details, duration, CGPA, and academic highlights.

4.7 Certifications and Achievements

Certifications are displayed in a responsive grid of badge-style cards, each showing the certification title, issuing organisation, year, and a link to the credential. Achievements are rendered as a styled list with violet bullet points, covering notable technical accomplishments including VyroOS (a complete operating system built from Assembly and C), VyroBrowser (an AI-powered desktop browser), and competitive programming activity.

4.8 Contact Form and Email Pipeline

The contact form represents the most technically complex section of the portfolio. It implements the following production-quality features: real-time field validation with touched-state tracking (errors shown only after a field is blurred), email format validation via regex, a character counter for the message field (warning at 800 characters, maximum at 1000), loading spinner state during submission, and animated success/error banners via `AnimatePresence`.

The form submits to a Next.js API Route (`/api/contact/route.ts`) via a JSON POST request. The route handler validates inputs server-side, instantiates the Resend client with a secret API key from environment variables, and sends a branded HTML email to the portfolio owner's Gmail address. The email includes a styled header, sender information table, message block with left violet border, and a one-click 'Reply to [sender]' CTA button.

```
const { data, error } = await resend.emails.send({
  from: 'Portfolio Contact <onboarding@resend.dev>',
  to: ['gauravganeshteegulla@gmail.com'],
  replyTo: senderEmail,
  subject: `New message from ${name}`,
  html: brandedEmailTemplate(name, email, message),
});
```

5. Animation and Motion Design

5.1 Framer Motion Architecture

Framer Motion's programming model centres on three primitives: motion components (HTML elements with animation superpowers), variants (named animation states), and the `AnimatePresence` wrapper (for exit animations on unmounting). These primitives compose naturally with React's declarative model and support orchestration via the `staggerChildren` and `delayChildren` variant properties.

Throughout the portfolio, a consistent animation vocabulary was established: sections enter from below ($y: 40 \rightarrow 0$) with an opacity fade ($0 \rightarrow 1$) over 0.5–0.6 seconds using an `easeOut` curve; cards within grids stagger with 70ms inter-item delays; hover interactions use `translate-y: -4px` to `-8px` with subtle box-shadow intensification. This consistency creates a coherent motion language that feels intentional rather than gratuitous.

5.2 Scroll-Triggered Reveals

Scroll-triggered animations are implemented using Framer Motion's `whileInView` prop combined with the `viewport={{ once: true }}` option. This triggers the animation exactly once when the element enters the viewport, preventing repetitive re-animation on scroll-back which would feel mechanical. The `AnimatedSection` wrapper component encapsulates this pattern, allowing any section to opt into scroll reveal with a single wrapper element.

```
export default function AnimatedSection({ children, id, className }) {
  return (
    <motion.section
      id={id}
      initial={{ opacity: 0, y: 40 }}
      whileInView={{ opacity: 1, y: 0 }}
      viewport={{ once: true, margin: '-80px' }}
      transition={{ duration: 0.55, ease: 'easeOut' }}
    >
      {children}
    </motion.section>
  );
}
```

5.3 Loader Animation

The loading sequence uses a character-level stagger pattern. The name string is split into an array of individual characters (including a space character handled separately). Each character is wrapped in a `motion.span` with: initial opacity 0, y-offset 40px, and blur-filter 8px; animated to opacity 1, y-offset 0, and blur-filter 0px; with a delay of $0.4 + (\text{index} * 0.05)$ seconds. This produces a left-to-right wave effect where each character cascades into place. The first six characters (the given name 'Gaurav') are rendered in white while remaining characters use the violet accent colour, creating a typographic rhythm.

5.4 Micro-interactions

Micro-interactions are the small, purposeful animations that respond to user actions and provide feedback. Throughout the portfolio, micro-interactions include: navigation links with a bottom-border reveal on hover; project cards lifting by 8px and intensifying their box-shadow on hover; social icons translating upward by 2px on hover; the submit button lifting by 2px and brightening on hover with a CSS transition; and the theme toggle icon crossfading between sun and moon symbols. These interactions are deliberately subtle — they should reward attention without demanding it.

6. Theming: Dark and Light Modes

6.1 CSS Custom Properties Strategy

CSS custom properties (variables) provide the foundation of the theming system. Rather than duplicating style rules for each theme, components reference semantic token names (`--text-primary`, `--bg-card`, `--border`) that are resolved at render time based on the active theme class. This indirection means that adding a third theme — or modifying an existing one — requires changing only the token definitions, not individual component styles.

The token vocabulary covers six concerns: background surfaces (primary, secondary, card), borders, text (primary, secondary, muted), and accent (violet, light violet, glow). This vocabulary was sufficient to theme all 15+ components in the portfolio without any component requiring hardcoded colour values for theme-sensitive properties.

6.2 next-themes Integration

The next-themes `ThemeProvider` is mounted at the root layout level, wrapping the entire application. It is configured with `attribute='class'` (applying the theme as a class on the element), `defaultTheme='light'` (light theme as the initial state for new visitors), and `enableSystem=false` (disabling automatic system preference detection in favour of explicit user choice).

The `useTheme` hook is consumed in the `Nav` component to power the toggle button, reading the current theme value and calling `setTheme` to switch between `'light'` and `'dark'`. The theme choice is persisted in `localStorage` under the key `'theme'` by next-themes, ensuring returning visitors see their preferred theme without flash.

6.3 Component-Level Token Usage

All components consume design tokens exclusively through Tailwind's arbitrary value syntax — e.g., `bg-[var(--bg-card)]`, `text-[var(--text-primary)]`, `border-[var(--border)]`. This approach preserves Tailwind's JIT compilation benefits while enabling dynamic theming. Hardcoded colour values are used only for elements that are intentionally theme-invariant: the violet accent colour (which remains constant across both themes), white text on violet backgrounds, and the gradient fill on the GG initials circle.

7. Performance Optimisation

7.1 Static Generation

The portfolio's root page is statically generated at build time by Next.js, producing a fully rendered HTML document that is served from Vercel's edge network with zero server-side computation on each request. Client-side JavaScript is loaded asynchronously and hydrates interactive components (animations, theme toggle, contact form) without blocking the initial render. This architecture achieves Time-to-First-Byte (TTFB) values under 50ms for users within Vercel's edge regions.

7.2 Image Optimisation

Project screenshots are served through Next.js's built-in `next/image` component, which automatically: converts PNG/JPG sources to WebP format (30–40% smaller); generates `srcset` attributes for responsive sizing; applies lazy loading via the browser's native `loading='lazy'` attribute; reserves layout space to prevent Cumulative Layout Shift (CLS); and serves images from Vercel's Image Optimisation API with aggressive CDN caching. The three featured project screenshots (1440x900px PNG, approximately 200–400KB each) are reduced to sub-100KB WebP files for typical viewport sizes.

7.3 Font Loading

Typography uses the Geist Sans and Geist Mono typefaces from Vercel's `next/font` package. `next/font` performs automatic font subsetting, self-hosts the font files alongside the application bundle (eliminating third-party font network requests), and injects optimised `font-display: swap` CSS to prevent invisible text during font loading. The font variables are applied to the root element and cascaded to all descendant components via CSS custom property references.

7.4 Bundle Analysis

The production build generates a Route (app) summary showing the root page as a static (SSG) route and `/api/contact` as a dynamic server route. The client JavaScript bundle is split automatically by Next.js's code splitting — Framer Motion (the largest dependency at ~50KB gzipped) is loaded only on pages that use it. The total first-load JS for the portfolio page is under 150KB gzipped, well within Google's recommended budget for mobile performance.

8. Deployment and DevOps

8.1 Vercel Deployment Pipeline

The portfolio is deployed on Vercel's Hobby tier, which provides: global CDN distribution across 100+ edge locations, automatic HTTPS with Let's Encrypt certificates, unlimited bandwidth on static assets, and 100GB of serverless function invocations per month — more than sufficient for a personal portfolio's contact form traffic. Build time averages 45–60 seconds, including TypeScript type-checking and static page generation.

8.2 Environment Variables

The single environment variable (`RESEND_API_KEY`) is stored in Vercel's encrypted environment variable store, scoped to the Production environment. It is never committed to the Git repository — the `.gitignore` configuration excludes all `.env*` files. A `.env.example` file with a placeholder value is committed to guide contributors through local setup.

8.3 Continuous Deployment via GitHub

The GitHub repository (github.com/Gaurav06120714/portfolio) is connected to the Vercel project. Every push to the main branch triggers an automatic production deployment. Pull request branches receive preview deployments at unique URLs, enabling visual review of changes before merge. This CI/CD pipeline eliminates manual deployment steps entirely — a git push is the sole deployment action required.

9. Testing and Quality Assurance

Quality assurance for this project was conducted across four dimensions: type safety, build integrity, cross-browser compatibility, and performance auditing.

9.1 TypeScript Compilation

TypeScript strict mode was enabled throughout the project. All components, hooks, data structures, and API route handlers are fully typed. The `npm run build` command runs the TypeScript compiler (`tsc`) before Next.js builds, ensuring that type errors block deployment. Over the course of development, several type errors were caught and resolved: the `Variants` import missing from `framer-motion`, incorrect prop types on custom components, and implicit any types in event handler callbacks.

9.2 Cross-Browser and Responsive Testing

The portfolio was tested across Chrome 124, Safari 17, Firefox 125, and Mobile Safari on iOS 17. CSS custom properties and Tailwind utilities used throughout the project have 96%+ global browser support. The one compatibility concern — CSS `backdrop-filter` (used for the nav blur effect) — has 93% support and degrades gracefully to a solid background in unsupported browsers.

Responsive behaviour was validated at five breakpoints: 375px (iPhone SE), 390px (iPhone 14), 768px (iPad), 1280px (laptop), and 1920px (desktop). The hamburger menu activates below the md breakpoint (768px); sidebars are hidden below md; project cards collapse from 12-column grid to full-width stacked layout; and the contact form collapses from two-column to single-column.

9.3 Performance Auditing

Lighthouse audits were run in Chrome DevTools on the production Vercel deployment in both mobile and desktop emulation modes. Key metrics observed:

- Performance: 94 (desktop), 88 (mobile)
- Accessibility: 96 — all images have alt text, interactive elements have aria-labels, colour contrast ratios meet WCAG AA
- Best Practices: 100 — HTTPS, no deprecated APIs, secure headers
- SEO: 100 — structured metadata, Open Graph tags, canonical URL

10. Results and Discussion

The completed portfolio achieves all primary objectives stated in Section 1.2. The site is live at `portfolio-five-roan-39.vercel.app`, loads in under 1.5 seconds on a 4G mobile connection, and delivers a visually cohesive experience across all tested devices and browsers.

10.1 Technical Outcomes

The most technically significant outcome is the contact pipeline architecture. By routing email delivery through a server-side Next.js API Route rather than a client-side service, the implementation achieves three goals simultaneously: API key security (never exposed in the browser), server-side input validation (prevents malformed requests from reaching the email service), and rich HTML email templating (impossible with client-side solutions). The Resend SDK's free tier (3,000 emails/month) is more than adequate for a personal portfolio's contact volume.

The screenshot capture pipeline using Puppeteer — automating a headless Chromium instance to visit running localhost applications and capture full-viewport PNG screenshots — proved a pragmatic solution to the challenge of showcasing live projects without requiring manual screenshot workflows. The tool identified an important edge case: projects with authentication-gated landing screens (VyroNotes required a backend login to proceed past a loading spinner) cannot be screenshotted in this automated fashion without session credentials.

10.2 Design Outcomes

The violet and indigo colour scheme creates a distinctive visual identity that is neither derivative of the reference site's green palette nor generic. The decision to use a light theme as the default — rather than the more common dark default for developer portfolios — differentiates the site immediately and improves readability in bright environments where the portfolio is most likely to be reviewed (office recruiters, daylight laptop screens).

The letter-by-letter name animation in the loader proved more impactful than a simple fade-in, creating a memorable first impression that establishes the site's motion vocabulary before any content is visible. The 50ms inter-character delay was calibrated through iterative testing — shorter delays lost the wave effect; longer delays felt slow and impatient.

10.3 Challenges Encountered

Several non-trivial challenges arose during development. The most significant was the removal of brand icons (GitHub, LinkedIn, Instagram) from `lucide-react` v1.17, which broke the icon imports across six components. The solution — creating a `SocialIcons.tsx` module with hand-crafted inline SVG components for each brand icon — required approximately 90 minutes of work but produced icons with pixel-perfect fidelity to the official brand assets.

TypeScript's strict mode surfaced several typing issues that would have been silent runtime errors in plain JavaScript: the `Variants` type missing from `Framer Motion` imports, `AnimatePresence` not included in component imports, and the viewport metadata API change in Next.js 14 requiring a separate `export const viewport` declaration. Each error was resolved before deployment, resulting in a zero-error production build.

11. Conclusion and Future Work

11.1 Conclusion

This paper has documented the complete design and implementation of a modern, production-grade developer portfolio website. The project demonstrates that open-source tooling — Next.js, TypeScript, Tailwind CSS, Framer Motion, and Resend — can be composed to produce a polished, performant, and fully functional portfolio site that competes aesthetically with commercially designed alternatives, at zero hosting cost and with full ownership of the codebase.

The architectural decisions made — static generation for performance, CSS custom properties for theming, server-side API routes for email security, and component-level animation via Framer Motion variants — are not specific to portfolio sites. They represent broadly applicable patterns for modern web application development that can be adapted to commercial projects, internal tools, and open-source products alike.

The portfolio is live, version-controlled, continuously deployed, and open-source at github.com/Gaurav06120714/portfolio — embodying the principle that a developer's portfolio should itself be an example of the quality of work they are capable of producing.

11.2 Future Work

Several enhancements are planned for future iterations:

- Blog integration — a `/blog` route using MDX for long-form technical writing, served as statically generated pages for maximum performance.
- CMS integration — replacing the hardcoded `portfolio.ts` data store with a headless CMS (Sanity or Contentful) to enable content updates without code changes.
- Custom domain — migration from the generated `vercel.app` subdomain to a personalised `.dev` domain for increased professional credibility.
- Analytics — integration of Vercel Analytics and Speed Insights to gather real-world performance data and visitor behaviour insights.
- Internationalisation — adding English/Hindi language support using Next.js's built-in `i18n` routing to broaden accessibility.
- Project detail pages — individual `/projects/[slug]` routes for featured projects with architecture diagrams, development timelines, and live demos.
- Profile photograph — replacing the GG initials gradient card with an optimised headshot to increase personal connection with visitors.

12. References

- [1] Vercel Inc. (2024). Next.js 14 Documentation — App Router. <https://nextjs.org/docs>
- [2] Microsoft Corporation. (2024). TypeScript Handbook. <https://www.typescriptlang.org/docs/>
- [3] Tailwind Labs. (2024). Tailwind CSS Documentation v3. <https://tailwindcss.com/docs>
- [4] Framer. (2024). Framer Motion API Reference. <https://www.framer.com/motion/>
- [5] Pacocoursey. (2024). next-themes: An abstraction for themes in your Next.js app. <https://github.com/pacocoursey/next-themes>
- [6] Resend. (2024). Resend Email API Documentation. <https://resend.com/docs>
- [7] Chiang, B. (2022). v4.brittanychiang.com — Personal Portfolio. <https://v4.brittanychiang.com>
- [8] Google. (2024). Lighthouse Performance Scoring. <https://developer.chrome.com/docs/lighthouse/performance/>
- [9] MDN Web Docs. (2024). CSS Custom Properties (Variables). https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties
- [10] Stack Overflow. (2024). Developer Survey 2024 — Most Popular Technologies. <https://survey.stackoverflow.co/2024/>
- [11] GitHub. (2024). Octoverse 2024 — State of Open Source. <https://octoverse.github.com/>
- [12] Puppeteer Team. (2024). Puppeteer Documentation. <https://pptr.dev/>
- [13] Google Fonts. (2024). Web Font Optimization. https://fonts.google.com/knowledge/using_type/using_web_fonts
- [14] W3C. (2023). Web Content Accessibility Guidelines (WCAG) 2.2. <https://www.w3.org/TR/WCAG22/>
- [15] Dillahunt, T. et al. (2018). Digital Self-Presentation in Technical Hiring. CHI 2018 Proceedings.
- [16] Vercel. (2024). Image Optimization in Next.js. <https://nextjs.org/docs/app/building-your-application/optimizing/images>
- [17] React Core Team. (2024). React Server Components — RFC. <https://github.com/reactjs/rfcs/blob/main/text/0188-server-components.md>
- [18] MDN. (2024). Intersection Observer API. https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API

About the Author

Gaurav Ganesh Teegulla is a Computer Science undergraduate specialising in Data Science at Sreenidhi Institute of Science and Technology, Hyderabad, India (Class of 2026). He builds full-stack web applications, AI/ML systems, and developer tools across the Vyro ecosystem — including VyroCoding (competitive coding platform), VyroMusic (AI-powered music platform), VyroPortify (AI portfolio generator), VyroBrowser (AI desktop browser), and VyroOS (operating system built from Assembly and C). He is an active competitive programmer on LeetCode and Codeforces, and a Data Science intern alumnus of SkillCraft Technology.

Email: gauravganeshteegulla@gmail.com

GitHub: github.com/Gaurav06120714

LinkedIn: linkedin.com/in/teegulla-gaurav-ganesh-6882b2380

LeetCode: leetcode.com/u/gauravganesh